

Déploiement d'un LLM Open-Source avec Docker Compose et Kubernetes

Rapport de projet DevOps

Introduction

Ce projet a pour objectif de déployer une application d'intelligence artificielle conteneurisée — un serveur de modèles de langage (**Ollama**) couplé à une interface web de conversation (**Open WebUI**) — d'abord localement avec Docker Compose, puis sur un cluster Kubernetes (Minikube), en appliquant les bonnes pratiques d'orchestration de conteneurs. Ce rapport présente l'architecture retenue, explique le rôle de chaque ressource Kubernetes créée, détaille le mécanisme de communication entre les services, justifie l'usage du stockage persistant et des sondes de santé, et conclut par une comparaison entre Docker Compose et Kubernetes.

1 Architecture de l'application

L'application repose sur deux services conteneurisés qui communiquent en réseau interne :

- **Ollama** : héberge et exécute les modèles de langage (`llama3.2:3b` et `mistral:7b` dans ce projet). Il expose une API HTTP sur le port `11434`.
- **Open WebUI** : interface web de type chat (similaire à ChatGPT), qui interroge l'API d'Ollama pour générer des réponses. Elle expose son interface sur le port `8080`.

Cette architecture a d'abord été validée avec Docker Compose (fichier `docker-compose.yaml`), puis reproduite intégralement avec des manifestes Kubernetes.

2 Déploiement Kubernetes : rôle de chaque ressource

Le déploiement Kubernetes est composé de neuf manifestes YAML, appliqués dans le namespace dédié `llm-devops`. Chaque ressource a une responsabilité précise.

2.1 Namespace

Le Namespace `llm-devops` isole l'ensemble des ressources du projet du reste du cluster. Il agit comme un espace de nommage logique : toutes les autres ressources (Deployments, Services, PVC, ConfigMap) y sont rattachées, ce qui simplifie la gestion, le nettoyage et évite les collisions de noms avec d'autres applications.

2.2 ConfigMap

Le ConfigMap `llm-config` centralise la configuration de l'application (URL interne d'Ollama, paramètres `OLLAMA_KEEP_ALIVE`, désactivation du téléchargement automatique des modèles d'embedding RAG, etc.). Les deux Deployments l'importent via `envFrom`. Cette approche remplace les variables d'environnement codées en dur : une modification de configuration ne nécessite pas de reconstruire les images ni de modifier les Deployments.

2.3 PersistentVolumeClaim (PVC)

Deux PVC ont été créés :

- `ollama-pvc` (15 Gi) : stocke les poids des modèles téléchargés (`/root/.ollama`).
- `webui-pvc` (2 Gi) : stocke la base de données d'Open WebUI (comptes utilisateurs, historique des conversations).

Voir la section 4 pour la justification détaillée de leur nécessité.

2.4 Deployments

Les **Deployments** `ollama` et `open-webui` définissent chacun un pod avec : l'image du conteneur, les points de montage vers les PVC, les variables issues du ConfigMap, les requêtes et limites de ressources CPU/mémoire, ainsi que les trois sondes de santé (voir section 5). Un Deployment garantit qu'un pod correspondant à cette spécification tourne en permanence : en cas de crash, Kubernetes le redémarre automatiquement, et en cas de mise à jour de configuration (`kubectl apply`), il orchestre un remplacement progressif du pod (rolling update) sans interruption de service.

2.5 Services

Deux **Services** de type `ClusterIP` (`ollama` et `open-webui`) exposent chaque Deployment via une adresse réseau stable. Leur rôle est détaillé en section 3.

2.6 Ingress

L'**Ingress** `open-webui-ingress` définit une route HTTP externe (`llm.local` → `open-webui:8080`) via un contrôleur `nginx`. Sur l'environnement de test (Minikube avec driver Docker sous Windows), l'accès direct à l'adresse du nœud n'étant pas routable depuis l'hôte sans configuration réseau supplémentaire (`minikube tunnel`), l'accès effectif pour les tests a été réalisé via `kubectl port-forward`, solution de repli explicitement acceptée par l'énoncé du projet. Le manifeste Ingress reste néanmoins fonctionnel et démontre la connaissance de ce mécanisme d'exposition.

3 Communication entre les services

Kubernetes attribue à chaque pod une adresse IP interne, mais celle-ci change à chaque redémarrage : elle ne peut donc pas être utilisée comme référence stable. Les **Services** résolvent ce problème en fournissant une adresse virtuelle stable (`ClusterIP`) associée à un nom DNS interne au cluster, résolu automatiquement par CoreDNS.

Concrètement, le Service `ollama` expose le pod Ollama sous le nom `ollama` (résolu en `ollama.llm-devops.svc.cluster.local`, ou simplement `ollama` au sein du même namespace). C'est exactement l'URL déclarée dans le ConfigMap :

```
OLLAMA_BASE_URL: "http://ollama:11434"
```

Le pod Open WebUI utilise cette variable pour contacter l'API d'Ollama, quelle que soit l'IP réelle du pod Ollama à un instant donné. Le Service fait office de load-balancer interne et de point d'indirection stable : si le pod Ollama est recréé (crash, mise à jour), son IP change mais le Service et son nom DNS restent identiques, garantissant la continuité de la communication. Ce mécanisme remplace le réseau bridge nommé (`networks: llm`) utilisé dans Docker Compose, où la résolution de noms de conteneurs jouait un rôle équivalent mais plus rudimentaire.

4 Pourquoi les PersistentVolumeClaims sont nécessaires

Par défaut, le système de fichiers d'un conteneur est **éphémère** : toute donnée écrite est perdue dès que le conteneur est recréé (redémarrage suite à un crash, mise à jour du Deployment, ou reprogrammation sur un autre nœud). Or deux types de données doivent impérativement survivre à ces événements :

- les poids des modèles LLM téléchargés dans Ollama (plusieurs gigaoctets, coûteux à retélécharger) ;
- la base de données d'Open WebUI (comptes utilisateurs, historique des conversations).

Un `PersistentVolumeClaim` découple le cycle de vie du stockage de celui du pod : les données sont écrites sur un volume physique (ici provisionné automatiquement par le `storage-provisioner` de Minikube) qui persiste indépendamment du pod qui l'utilise. Ce comportement a été vérifié concrètement au cours du projet : lors de l'ajustement des limites de ressources du Deployment Ollama (voir section 6), le pod a été recréé plusieurs fois, et les modèles `llama3.2:3b` et `mistral:7b` sont restés disponibles sans nouveau téléchargement, confirmant le bon fonctionnement du PVC.

5 Rôle des sondes de santé (probes)

Chaque conteneur des Deployments est équipé de trois sondes complémentaires, interrogeant un point de terminaison HTTP (`/api/version` pour Ollama, `/health` pour Open WebUI) :

- **Startup Probe** : vérifie que l'application a fini son démarrage avant que les autres sondes ne soient prises en compte. Elle tolère un délai de démarrage long (jusqu'à 150 secondes ici) sans déclencher de redémarrage prématuré du conteneur — utile car le serveur Ollama ou Open WebUI peut mettre du temps à devenir disponible au premier lancement.
- **Readiness Probe** : détermine si le pod est prêt à recevoir du trafic. Si elle échoue, le pod est retiré des endpoints du Service (le trafic n'y est plus routé) sans que le conteneur soit redémarré — utile lors d'une surcharge temporaire.
- **Liveness Probe** : détermine si le conteneur est toujours fonctionnel. Si elle échoue de façon répétée, Kubernetes redémarre le conteneur, permettant de récupérer automatiquement d'un blocage applicatif (deadlock, fuite mémoire).

Ces sondes ne sont pas restées théoriques dans ce projet : le pod Open WebUI a initialement échoué sa `startupProbe` car l'application tentait de télécharger un modèle d'embedding HuggingFace (`sentence-transformers/all-MiniLM-L6-v2`) au démarrage, opération plus longue que le délai initialement prévu. La correction a consisté à désactiver ce téléchargement non nécessaire pour ce projet (`RAG_EMBEDDING_MODEL_AUTO_UPDATE=false`) plutôt qu'à simplement allonger le délai, ce qui illustre l'utilité des probes pour détecter un comportement de démarrage anormal.

6 Gestion des ressources : retour d'expérience

La définition des `requests` et `limits` CPU/mémoire a nécessité plusieurs itérations, ce qui a permis de mieux comprendre leur rôle :

- **OOMKilled (mémoire insuffisante)** : la limite mémoire initiale d'Open WebUI (512 Mi) était trop basse et provoquait un `CrashLoopBackOff` (`OOMKilled`, code de sortie 137). Portée à 2 Gi, le pod a démarré normalement.
- **Limite CPU trop restrictive** : le modèle `mistral:7b` (7 milliards de paramètres, inférence CPU) ne produisait aucune réponse après plus de dix minutes avec une limite de 2 vCPU. Le noeud disposant de 8 coeurs logiques, la limite a été relevée à 6 vCPU, ramenant le temps de première réponse à moins de 70 secondes (chargement du modèle inclus).

Ces incidents illustrent concrètement pourquoi les `requests` (ressources garanties, utilisées pour l'ordonnancement) et les `limits` (plafond dur, au-delà duquel le conteneur est limité ou tué) doivent être dimensionnées en fonction de la charge réelle de l'application, et non de valeurs arbitraires.

Aspect	Docker Compose	Kubernetes
Portée Déclaration	Un seul hôte Docker Un seul fichier YAML	Cluster de plusieurs noeuds Plusieurs manifestes granulaires (Namespace, Deployment, Service, etc.)
Résilience	Redémarrage simple (restart: unless-stopped)	Auto-guérison avancée via Deployments, ReplicaSets et probes
Mise à l'échelle	Manuelle, limitée (-scale)	Native, avec support d'auto-scaling (HPA)
Réseau	Réseau bridge nommé, résolution par nom de conteneur	Services + CoreDNS, load-balancing interne
Stockage	Volumes Docker nommés	PersistentVolumeClaims avec provisionnement dynamique
Santé	Un seul healthcheck par service	Trois sondes distinctes (startup/readiness/liveness) aux rôles différenciés
Cas d'usage	Développement local, prototypage	Production, haute disponibilité, orchestration à grande échelle

Table 1: Comparaison synthétique entre Docker Compose et Kubernetes

7 Différences entre Docker Compose et Kubernetes

En résumé, Docker Compose reste pertinent pour un développement local rapide sur une seule machine, tandis que Kubernetes apporte les mécanismes nécessaires à une exploitation en production : résilience automatisée, découplage explicite des responsabilités, gestion fine des ressources et capacité de montée en charge.

8 Validation du déploiement

Le déploiement a été validé selon les critères suivants, conformément aux exigences du projet :

- les deux pods sont à l'état **Running** (voir figure 1);
- aucun pod n'est en **CrashLoopBackOff** ni en **Pending** ;
- Open WebUI communique correctement avec Ollama via le Service interne ;
- la page d'accueil d'Open WebUI est accessible via le déploiement Kubernetes (figure 2) ;
- les modèles **llama3.2:3b** et **mistral:7b** sont disponibles et répondent aux invites (figures 3 et 4).



```

PS C:\Users\Fasyakadiou\Documents\HORS_INTOUCH\dic2-2026\llm-devops-project> kubectl get pods -n llm-devops
NAME                                READY   STATUS    RESTARTS   AGE
ollama-5c67676bd9-kx9zd             1/1     Running   1 (4m28s ago)   33m
open-webui-5b754d7d56-26djs         1/1     Running   1 (35m ago)     4h34m
PS C:\Users\Fasyakadiou\Documents\HORS_INTOUCH\dic2-2026\llm-devops-project> kubectl get svc -n llm-devops
NAME      TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
ollama    ClusterIP   10.108.40.128 <none>        11434/TCP   5h4m
open-webui ClusterIP   10.107.116.65 <none>        8080/TCP    5h4m
PS C:\Users\Fasyakadiou\Documents\HORS_INTOUCH\dic2-2026\llm-devops-project>

```

Figure 1: Pods et Services Kubernetes en fonctionnement (`kubectl get pods` / `kubectl get svc`)

9 Fonctionnalités bonus

Au-delà des exigences obligatoires, quatre améliorations supplémentaires ont été implémentées : la restructuration des manifestes avec Kustomize (environnements dev/prod), des NetworkPolicy, des PodDisruptionBudget, et un pipeline de validation continue (CI) sur GitHub Actions. Pour ces raisons de robustesse, aucune de ces fonctionnalités n'a été appliquée directement sur le cluster Minikube déjà validé (sections précédentes) : le risque de saturer à nouveau les ressources de la

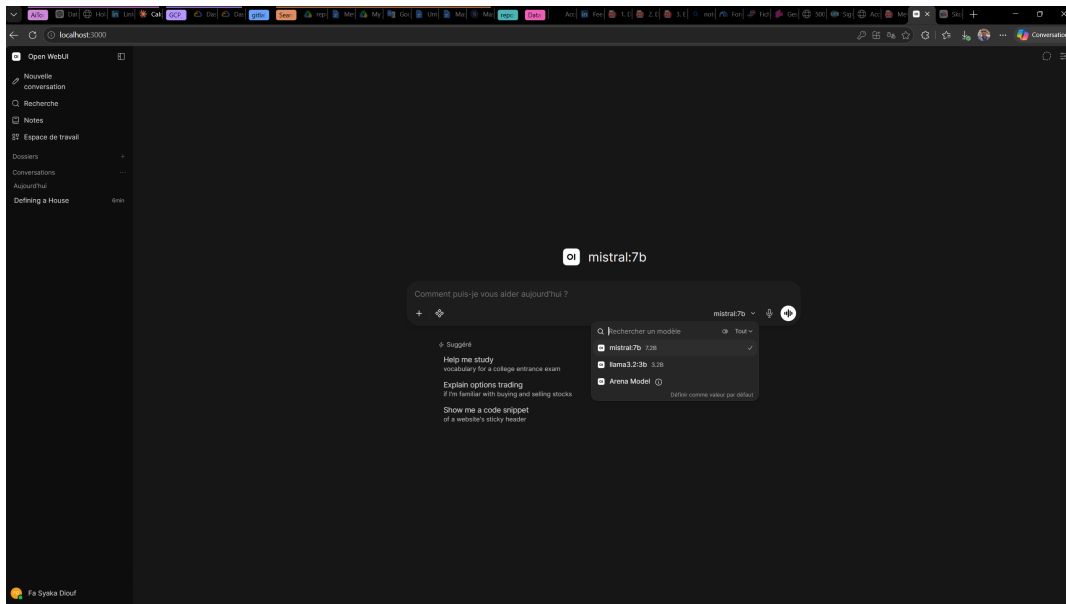


Figure 2: Page d'accueil d'Open WebUI, servie par le déploiement Kubernetes

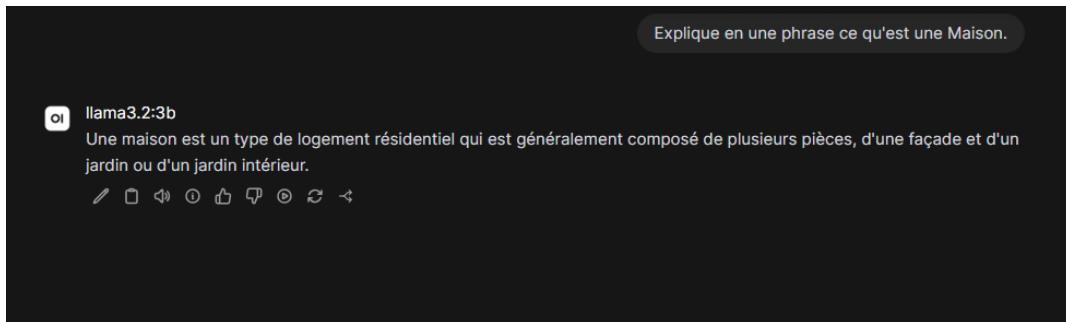


Figure 3: Réponse générée par le modèle llama3.2:3b

machine (voir section 6) n'en valait pas la peine. Chaque élément a en revanche été rendu et validé localement (`kubectl kustomize + kubeconform`), puis vérifié en conditions réelles via le pipeline CI décrit en 9.4.

9.1 Restructuration Kustomize (environnements dev / prod)

Les neuf manifestes de base ont été déplacés tels quels dans `k8s/base/`, accompagnés d'un fichier `kustomization.yaml` qui les référence. Deux overlays ont ensuite été créés :

- `k8s/overlays/dev/` : espace de noms `llm-devops-dev`, limites CPU/mémoire d'Ollama réduites (2 vCPU / 3 Gi au lieu de 6 vCPU / 5 Gi), hôte d'Ingress `llm-dev.local` — adapté à une itération rapide sur une machine peu puissante.
- `k8s/overlays/prod/` : espace de noms `llm-devops-prod`, stockage Ollama porté à 30 Gi, et ajout des NetworkPolicy et PodDisruptionBudget détaillés ci-dessous.

Chaque overlay ne fait que patcher la base commune : un correctif appliqué dans `base/` profite automatiquement aux deux environnements, sans duplication de YAML. Les deux rendus ont été validés avec `kubeconform`, qui vérifie leur conformité au schéma de l'API Kubernetes :

```
$ kubectl kustomize k8s/overlays/dev | kubeconform -strict -summary
Summary: 9 resources found in 1 file - Valid: 9, Invalid: 0, Errors: 0, Skipped: 0

$ kubectl kustomize k8s/overlays/prod | kubeconform -strict -summary
Summary: 13 resources found in 1 file - Valid: 13, Invalid: 0, Errors: 0, Skipped: 0
```

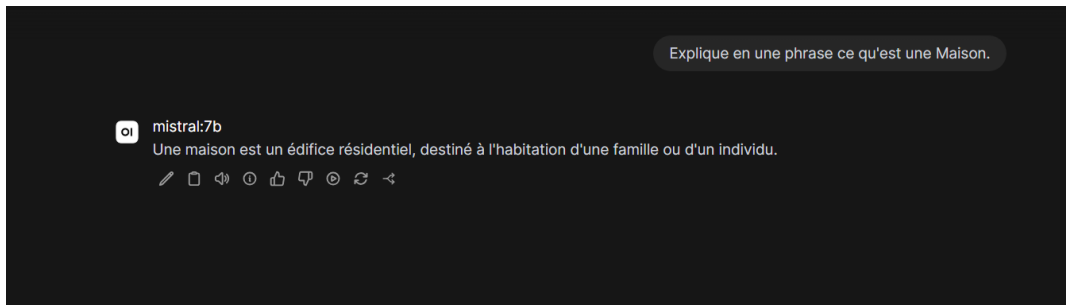


Figure 4: Réponse générée par le modèle mistral:7b

9.2 NetworkPolicy

Deux règles réseau, actives uniquement dans l'overlay **prod** :

- Ollama n'accepte du trafic entrant que depuis les pods portant le label `app.kubernetes.io/name: open-webui`, sur le port 11434.
- Open WebUI n'accepte du trafic entrant que depuis le namespace du contrôleur Ingress (`ingress-nginx`), sur le port 8080.

Limite honnête à signaler : une NetworkPolicy n'est effective que si le plugin réseau (CNI) du cluster l'implémente. Le CNI par défaut de Minikube observé ici (`kindnet`, visible dans les pods du namespace `kube-system`) accepte ces objets sans erreur mais ne les fait pas respecter. Ces manifestes sont donc corrects et fonctionneraient tels quels sur un cluster de production utilisant un CNI compatible (Calico, Cilium, Weave), mais leur effet n'a pas pu être vérifié en conditions réelles sur cet environnement de test.

9.3 PodDisruptionBudget

Un `PodDisruptionBudget` avec `minAvailable: 1` a été défini pour chacun des deux Deployments (overlay **prod**). Il protège contre les interruptions volontaires (par exemple un `kubectl drain` lors d'une maintenance planifiée d'un nœud) : Kubernetes refusera d'évincer un pod si cela faisait tomber le nombre de répliques disponibles en dessous du seuil défini. Ce mécanisme est complémentaire aux probes, qui gèrent les pannes non planifiées.

9.4 Pipeline GitHub Actions

Pour valider ce travail dans des conditions représentatives, le contenu de `llm-devops-project` a été extrait dans un dépôt public dédié : github.com/Fa-Syaka-diouf/llm-devops-project. Un pipeline GitHub Actions (`.github/workflows/k8s-validate.yml`) s'y déclenche à chaque modification du dossier `k8s/` : il installe `kubectl`, rend les deux overlays avec Kustomize, les valide avec `kubeconform`, puis vérifie le style des fichiers YAML avec `yamllint`. L'ensemble tourne sur l'infrastructure de GitHub, sans aucune charge sur la machine locale.

Le premier déclenchement du pipeline a d'ailleurs révélé une vraie erreur (fichiers YAML sans retour à la ligne final), corrigée immédiatement — preuve que le pipeline fonctionne réellement et ne se contente pas de toujours réussir. Après correction, le run suivant est passé avec succès :

```
Run: https://github.com/Fa-Syaka-diouf/llm-devops-project/actions/runs/34655853154
Job "validate" : succeeded in 11s

Summary: 9 resources found in 1 file - Valid: 9, Invalid: 0, Errors: 0
Summary: 13 resources found in 1 file - Valid: 13, Invalid: 0, Errors: 0
yamllint: outcome=success
```

Conclusion

Ce projet a permis de mettre en pratique l'ensemble du cycle de conteneurisation d'une application d'IA, depuis un déploiement local avec Docker Compose jusqu'à une orchestration Kubernetes conforme aux bonnes pratiques de production (isolation par namespace, configuration externalisée, stockage persistant, gestion explicite des ressources, sondes de santé multi-niveaux). Les difficultés rencontrées en conditions réelles — limite mémoire trop basse, quota CPU insuffisant pour l'inférence d'un modèle de 7 milliards de paramètres, téléchargement bloquant au démarrage — ont constitué des cas d'étude concrets pour comprendre le rôle exact de chaque mécanisme Kubernetes plutôt qu'une simple application théorique du cours.